



Projet Conception et Réalisation de Systèmes Électroniques: Ohmmètre Digital

I/Introduction.....	2
II/ Synoptique et description du système.....	2
1) Conditionnement.....	2
2) Amplification.....	4
III/ Montage complet.....	5
IV/ Le code Arduino.....	6
V/ Tests et résultats.....	7
1) Simulations sur TinkerCad.....	7
2) Tests sur notre carte PCB.....	8
VI/ Conclusion.....	9
VII/ Sources.....	9
VIII/ Annexes.....	10

I/Introduction

Ce projet consiste à concevoir un ohmmètre numérique capable de mesurer une résistance et d'afficher sa valeur sur un écran LCD. Le cahier des charges impose une plage de mesure de $200\ \Omega$ à $20\ \text{k}\Omega$, avec une précision d'au moins 1 % sur l'ensemble de la gamme. Le système doit s'appuyer sur une carte Arduino Nano pour l'acquisition et le traitement du signal. C'est aussi cette carte qui va alimenter en +5V le système entier. Afin de répondre à ces exigences de précision et de plage de mesure, il est nécessaire de concevoir une chaîne de traitement adaptée. Avant de présenter le montage complet, nous détaillons d'abord l'architecture générale du système ainsi que les choix technologiques retenus, en commençant par le synoptique.

II/ Synoptique et description du système

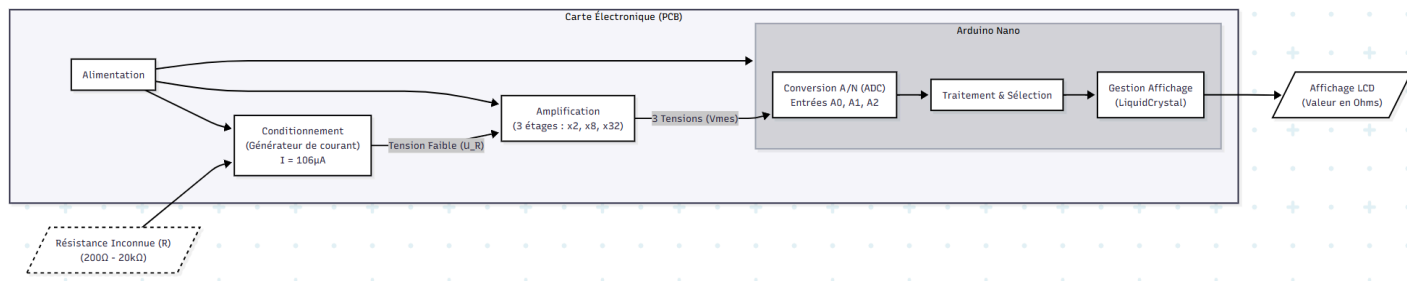


Figure 1: synoptique

1) Conditionnement

La partie conditionnement a été réalisée sous la forme d'un générateur de courant utilisant un amplificateur opérationnel (AOP) et un réseau de résistances. Nous réalisons une source de courant afin d'imposer un courant constant et parfaitement connu dans la résistance à mesurer. Cela permet d'obtenir une tension strictement proportionnelle à la résistance, ce qui garantit une mesure précise sur toute la plage nécessaire ($200\ \Omega$ à $20\ \text{k}\Omega$). Ce montage permet de générer une intensité indépendante de la résistance en sortie:

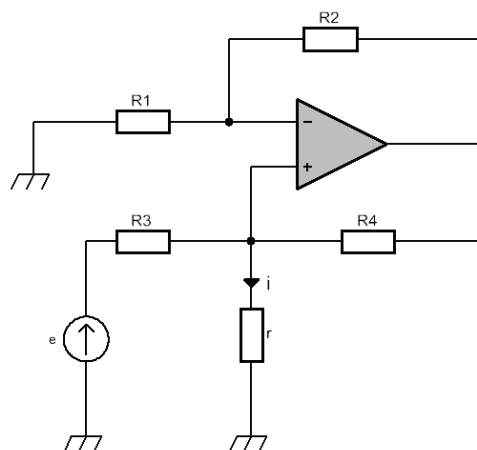


Figure 2: Schéma du montage pour le conditionnement

Pour assurer ce fonctionnement, il est nécessaire de respecter la condition :

$$R_1 R_4 = R_2 R_3$$

Cette condition est satisfaite dans notre cas, puisque nous avons choisi les quatre résistances identiques, chacune de valeur $47k\Omega$. L'intensité obtenue est alors donnée par la relation :

$$i = \frac{e}{R} = \frac{5}{47\,000} = 0,106\,mA$$

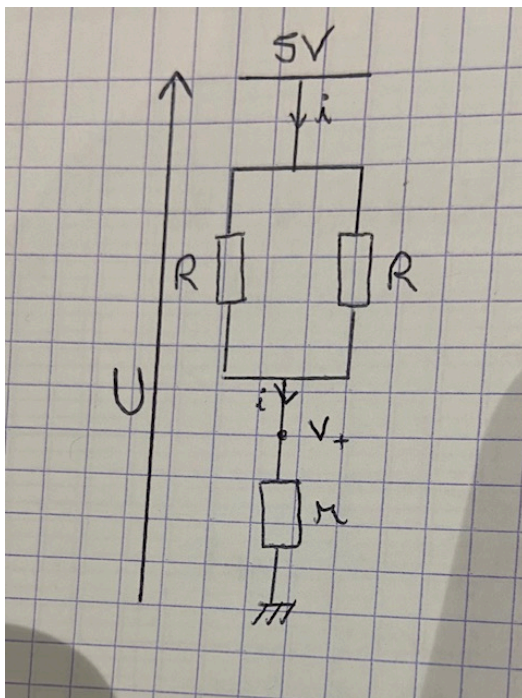
où e est la tension issue du pin 5V de l'arduino et R la valeur des résistances.

Afin de valider le fonctionnement du générateur de courant, un ampèremètre a été inséré en série dans le circuit. La mesure a indiqué un courant de sortie de $0,108\,mA$. Ceci est donc en accord avec la théorie en tenant compte du fait que les résistances ne sont pas exactement identiques en pratique.

Choix du dimensionnement:

Le choix de dimensionnement des résistances à $47k\Omega$ a été compliqué. Nous sommes d'abord partis avec des résistances de $22k\Omega$. Cependant, dans ce cas, l'AOP atteignait sa tension de saturation ($\approx 5\,V$), ce qui empêchait le fonctionnement linéaire du générateur : le courant dépendait alors de la résistance mesurée, ce qui est incompatible avec un ohmmètre précis. Nous avons mesuré la tension à différents points pour vérifier que l'AOP ne fonctionnait pas en mode linéaire car $\varepsilon \neq 0$.

Pour éviter cette saturation, nous avons étudié le cas limite où l'AOP délivre sa tension maximale. Nous avons donc fait le schéma équivalent dans le cas où la tension de sortie de l'ALI est maximale (à $5V$) pour déterminer les résistances à prendre afin de pouvoir réaliser la source de courant peu importe la résistance à mesurer. Dans ce cas, $r = R_{max}$.



$$\text{Il faut } r \geq R_{\text{parallèle}} \text{ or } R_{\text{parallèle}} = \frac{R}{2}.$$

$$\text{On effectue la loi d'Ohm: } U = i(R_{\text{parallèle}} + R_{\text{max}}).$$

Or on a toujours en fonctionnement linéaire:

$$i = \frac{e}{R} = \frac{e}{2R_{\text{parallèle}}} = \frac{U}{2R_{\text{parallèle}}} \text{ car } e=5V=U.$$

Dans ce cas en comparant la loi d'Ohm et cette égalité on a:

$$2iR_{\text{parallèle}} = i(R_{\text{parallèle}} + R_{\text{max}})$$

donc on en déduit que $R_{\text{parallèle}} = R_{\text{max}}$ et donc vu l'inégalité de base: **$R \geq 40k\Omega$.**

Le choix de résistances de 47 kΩ permet donc d'assurer le fonctionnement linéaire du montage pour toute la plage de résistances à mesurer, sans saturer l'AOP.

2) Amplification

Cependant, avec un si petit courant les plus faibles résistances ne pourront être mesurer en garantissant la précision à 1%. En effet, certaines tensions sont trop petites pour être converties avec précision par l'ADC de l'Arduino.

Pour garantir une précision de l'ordre de 1 %, le signal doit rester suffisamment au-dessus du bruit et de la résolution de mesure. Nous imposons ainsi que la tension minimale soit d'au moins : 0,7V. Nous allons donc avoir besoin d'amplifier le signal pour pouvoir répondre à toute la plage de résistances. Nous avons fait le choix d'amplifier 3 fois le signal avec un gain de 2 et deux gains de 4 ce qui nous permet de mesurer les résistances de 200Ω à 20kΩ. Plus précisément, trois sorties différentes seront branchées à l'Arduino Nano, une à la sortie du gain de 2 pour mesurer les plus grandes résistances, une à la sortie de la deuxième amplification de $2 \times 4 = 8$ pour les résistances intermédiaires et la dernière à la sortie de la troisième amplification de $2 \times 4 \times 4 = 32$ pour les plus petites valeurs de résistance. Nous sommes partis sur un montage basique d'amplificateur non inverseur:

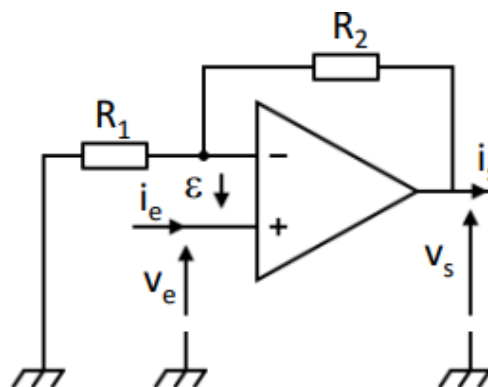


Figure 3: Schéma du montage amplificateur non inverseur

En appliquant un diviseur de tension nous trouvons que: $\frac{v_s}{v_e} = 1 + \frac{R_2}{R_1}$

Ainsi il suffit de prendre un rapport de 1 ou de 3 environ entre les 2 résistances pour avoir une amplification de 2 ou de 4. Nous avons pris $R_2 = 12\text{k}\Omega$ et $R_1 = 12\text{k}\Omega$ pour le gain de 2 et $R_2 = 12\text{k}\Omega$ et $R_1 = 3,9\text{k}\Omega$ pour le gain de 4.

Calibre	Plage de résistances mesurables
x32	200Ω à 800Ω
x8	800Ω à 2,5kΩ
x2	2,5kΩ à 20kΩ

Figure 4: Tableau récapitulatif des calibres

L'Arduino lit les trois tensions simultanément et sélectionne automatiquement la voie non saturée offrant le gain le plus élevé, ce qui maximise la précision de la mesure.

III/ Montage complet

En complément du conditionnement et des étages d'amplification, nous avons également câblé un écran LCD sur la carte Arduino Nano, conformément au tutoriel d'utilisation. Cet écran permettra d'afficher la valeur de la résistance une fois le code de calcul implémenté.

Le schéma suivant présente l'implémentation complète du circuit réalisée sous KiCad, intégrant la source de courant, les amplificateurs et l'interface avec l'Arduino. À partir de ce schéma, nous avons conçu notre propre carte PCB, puis réalisé manuellement le perçage et les soudures afin d'obtenir un montage final propre et utilisable.

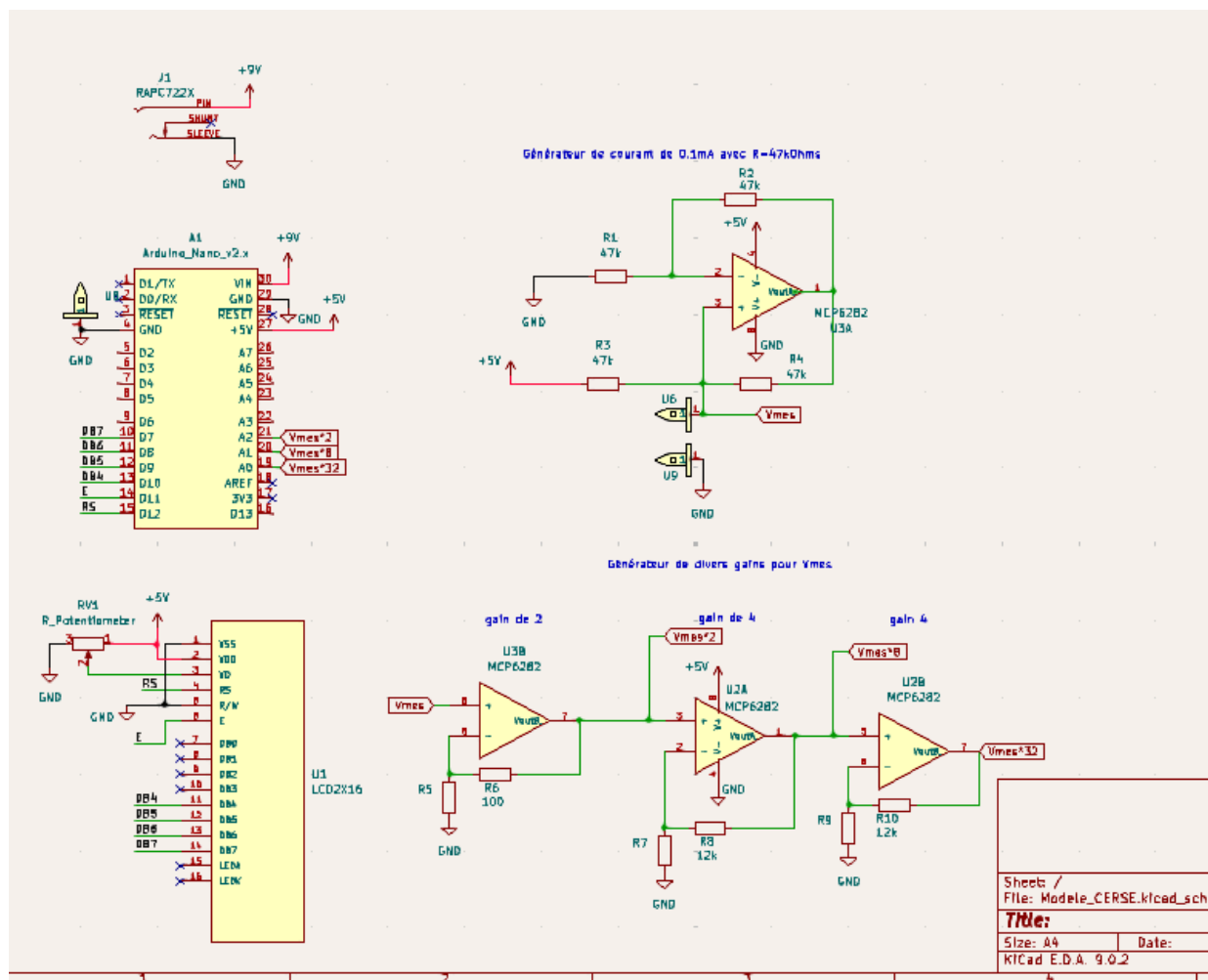


Figure 5 : Schéma Kicad

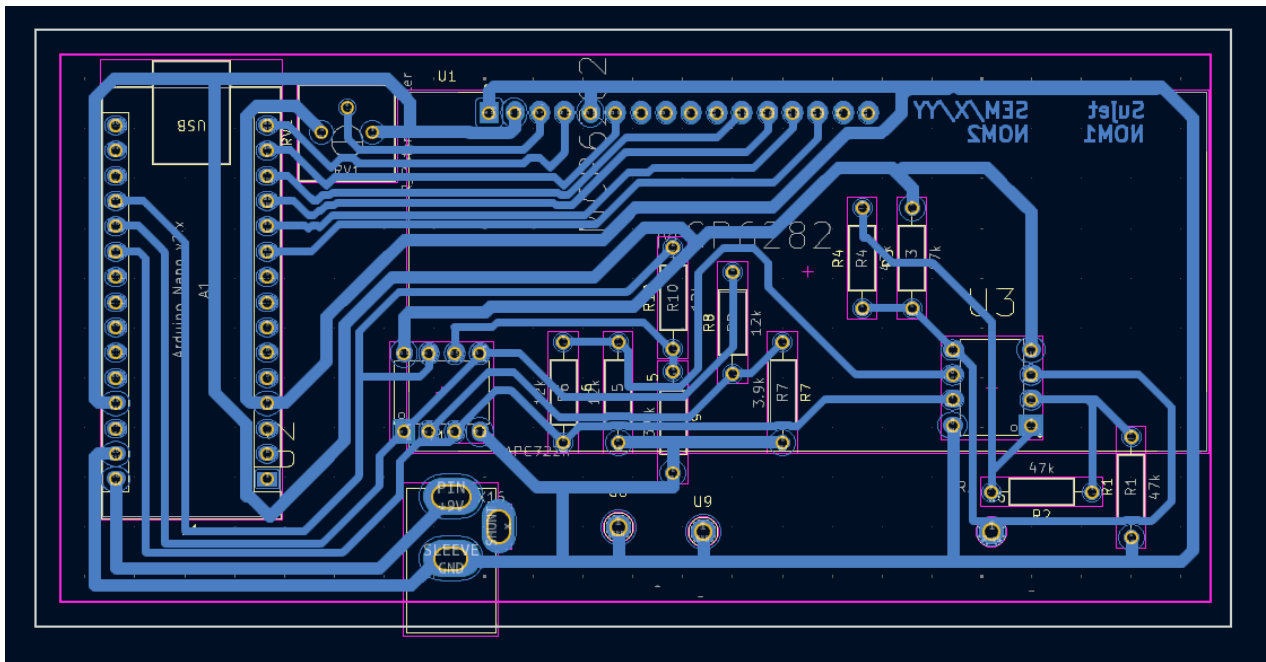


Figure 6 : Schéma de la carte PCB

IV/ Le code Arduino

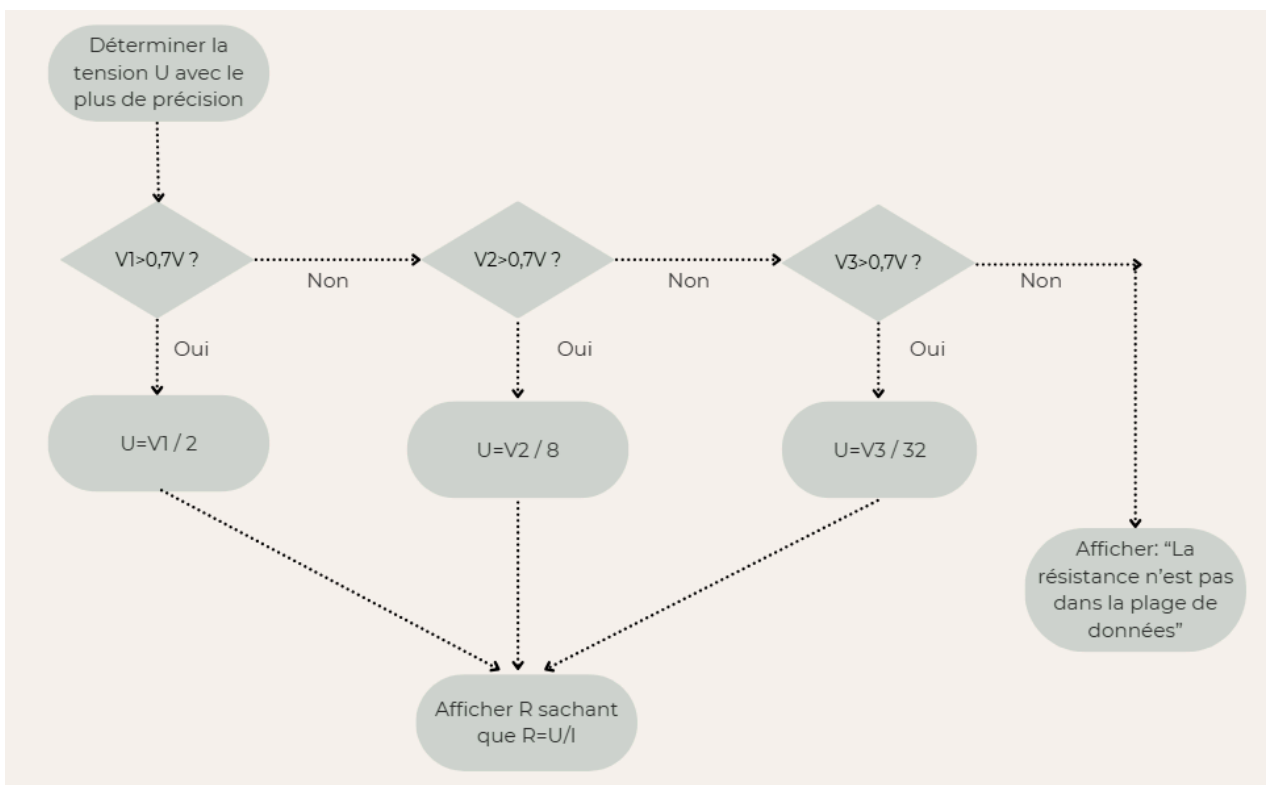


Figure 7 : Algorithme de programmation

V/ Tests et résultats

1) Simulations sur TinkerCad

Nous avons décidé de réaliser notre schéma sur TinkerCad, ce qui nous a permis de tester différentes solutions sans risque et de pouvoir tester et simuler des scénarios depuis chez nous. Voici le schéma final:

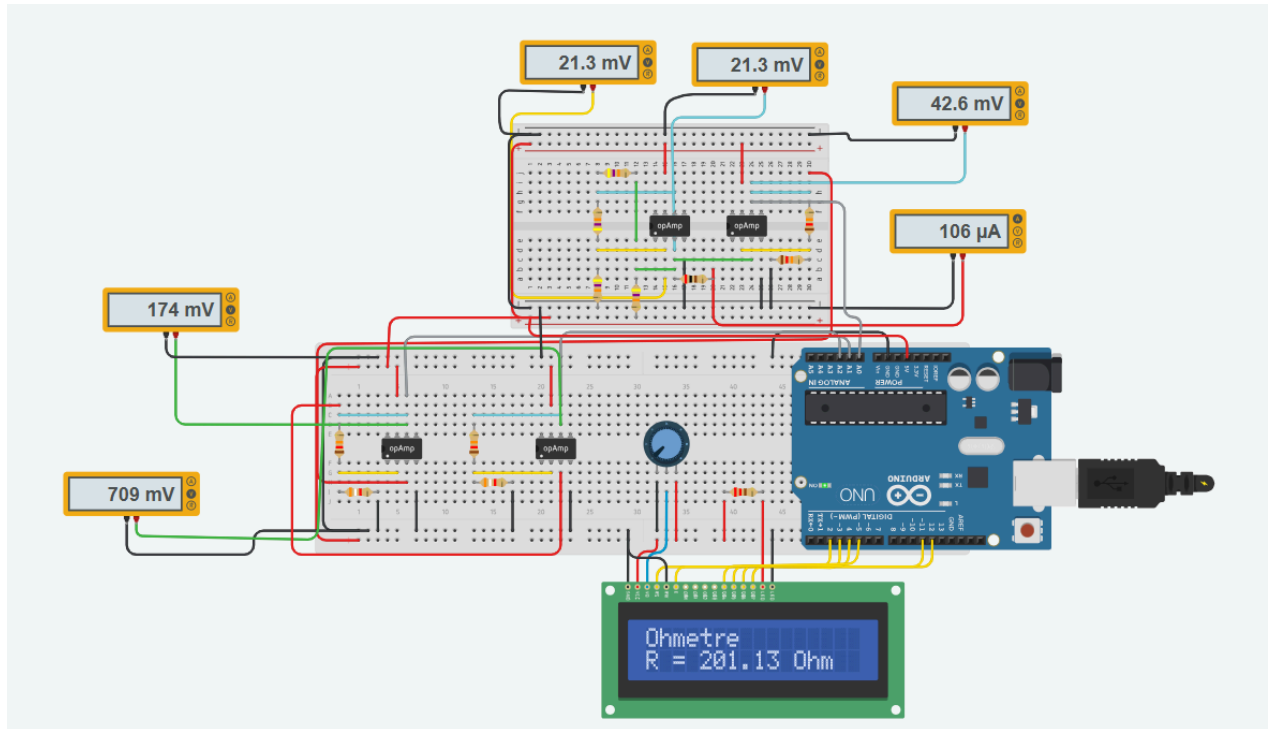


Figure 8 : schéma sur Tinkercad

Résistance théorique (en Ohm) ▾	Résistance mesurée par notre système Tinkercad (en Ohm) ▾	Précision (en pourcentage) ▾
200	201.13	0.565
500	502.12	0.424
1000	1006.68	0.668
1500	1504.36	0.2906666667
2000	2007.7	0.385
5000	5025.91	0.5182
10000	10028.77	0.2877
15000	15054.69	0.3646
20000	20080.6	0.403

Figure 9 : tests de la simulation

Après avoir fait les tests, nous nous sommes rendu compte que théoriquement notre montage était fonctionnel puisque la simulation de Tinkercad nous permet de mesurer la valeur de la résistance avec un pourcentage d'erreurs toujours inférieur à 1%. Nous avons donc pu confirmer que notre source de courant était efficace puisque en variant la résistance tout au long de la plage de données, nous avons toujours la

même intensité. Nous avons aussi pu tester nos calibres issus des amplifications. On voit bien que même dans le pire scénario ($r=200\Omega$), une tension est supérieure à 0,7V, celle multipliée 32 fois. Ainsi, TinkerCad montre que le système que nous avons conçu respecte l'ensemble des contraintes du cahier des charges et répond pleinement à la problématique posée dans ce bureau d'étude.

2) Tests sur notre carte PCB

Néanmoins, sur notre carte PCB nous ne sommes pas parvenus à répondre entièrement aux consignes puisqu'on ne trouve pas une valeur de résistance cohérente à chaque calibre. L'entrée amplifiée 32 fois a été testé avec une résistance de 463 ohm et a pu être mesuré par notre montage avec une erreur inférieure à 1%. Notre montage est donc perfectible mais il valide le principe de fonctionnement global de l'ohmmètre et que la partie conditionnement et amplification fonctionne correctement sur la plage la plus sensible. Les écarts observés sur les autres voies sont probablement dus à un problème de câblage, de soudure ou de surtension que la simulation TinkerCad n'a pas pu révéler.

Quelques améliorations sont possibles pour clôturer ce travail. Les écarts importants observés sur les voies d'amplification à gains inférieurs (x2 et x8) lors des tests réels sont probablement dus à des problèmes de câblage, de soudure ou de parasites. Une révision de la conception du circuit imprimé est recommandée. De plus, Le code actuel utilise des seuils fixés pour la sélection de gain (par exemple, 0,7V pour la tension minimale et 4,8V pour la saturation). Ces seuils pourraient être ajustés et calibrés plus précisément en fonction de la résolution réelle de l'ADC de l'Arduino, et des tensions de saturation précises des AOP, afin d'optimiser le changement de calibre et d'éviter les valeurs fausses.

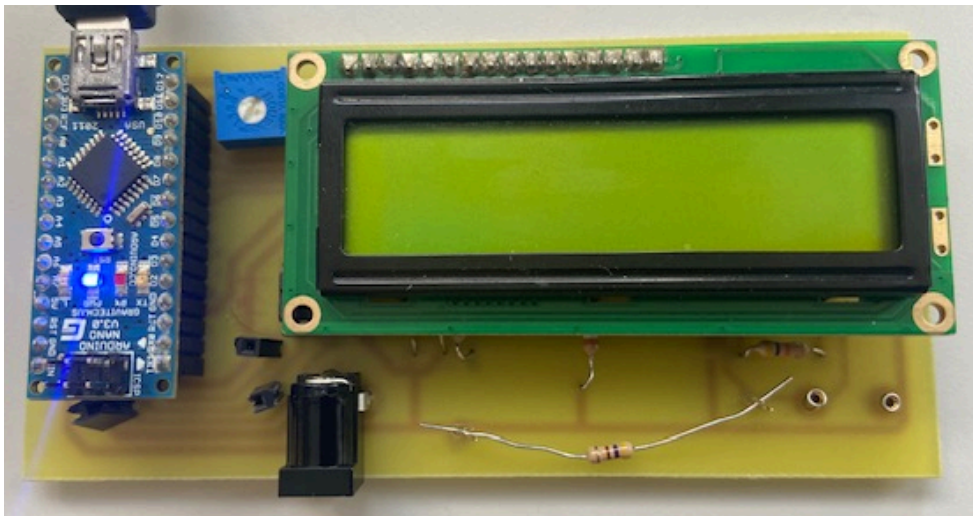


Figure 10 : carte PCB avec écran

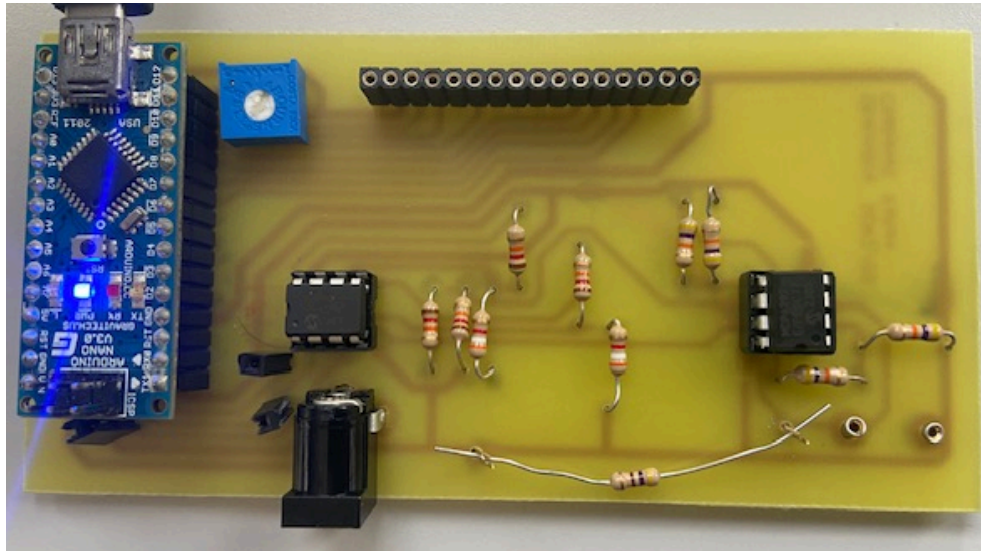


Figure 11 : carte PCB sans écran

Résistance mesurée par ohmmètre (en ohm) ▾	Résistance mesurée par notre carte(en Ohm) ▾	Précision (en pourcentage) ▾
325	323.4	0.4923076923
463	461.9	0.2375809935
1188	1300	9.427609428
3346	22000	557.5014943
12032	22000	82.84574468
14080	22000	56.25

Figure 12 : tests du montage réel

VI/ Conclusion

Ce projet autour de l'ohmmètre digital nous a permis de concevoir puis de tester une chaîne complète de mesure reposant sur une source de courant stable, suivie d'étages d'amplification adaptés à différentes gammes de résistance. Les simulations réalisées sur TinkerCad ont validé notre architecture, avec une précision systématiquement inférieure à 1 %, conformément au cahier des charges. En pratique, si certaines voies amplifiées présentent encore des écarts dus à un câblage ou un code perfectible, la gamme la plus sensible ($\times 32$) a donné des résultats fiables, démontrant le bon fonctionnement du principe de mesure. L'intégration de l'écran LCD et du code Arduino constitue également une base solide pour l'affichage final de la résistance. Ainsi, même si des ajustements restent nécessaires pour obtenir un dispositif totalement opérationnel sur toute la plage $200\ \Omega - 20\ \text{k}\Omega$, le projet valide le dimensionnement, le fonctionnement général et la pertinence des choix technologiques retenus. Cependant, quelques améliorations permettraient de conclure parfaitement ce travail comme la révision de la carte, des soudures et l'analyse plus approfondis de la saturation des AOP pour déterminer au mieux les seuils de tension.

VII/ Sources

Convertisseur tension courant (générateur de courant),

http://electronique.aop.free.fr/AOP_lineaire_NF/9_convertisseurTC.html

Générateur de courant constant,

https://www.sonelec-musique.com/electronique_bases_gene_courant.html

VIII/ Annexes

lien TinkerCad:

<https://www.tinkercad.com/things/bPSrHTqUKqh-epic-uusam-kieran/editel?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard>

Code:

```
#include <LiquidCrystal.h>

#define input_3 A0
#define input_2 A1
#define input_1 A2

float i = 0.000106; // courant en ampères (A)

// LCD
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup() {
    lcd.begin(16, 2);
    lcd.print("Ohmetre");
    Serial.begin(9600);
}

void loop() {
    // Mesure des tensions analogiques
```

```
float V1 = analogRead(input_1) * (5.0 / 1023.0);
float V2 = analogRead(input_2) * (5.0 / 1023.0);
float V3 = analogRead(input_3) * (5.0 / 1023.0);

// Calcul de la tension utile (selon ta logique de sélection)
float Vmes = gestion_input(V1, V2, V3);

// Calcul de la résistance
float R = Vmes / i;

// Affichage
lcd.setCursor(0, 1);
lcd.print("R = ");
lcd.print(R);
lcd.print(" Ohm      ");

Serial.print("R = ");
Serial.println(R);

delay(500);
}

float gestion_input(float V1, float V2, float V3) {
    float seuil = 4.8; // seuil de saturation (V)

    // Si une tension est saturée, on la met à 0 pour l'ignorer
    if (V1 >= seuil) V1 = 0;
    if (V2 >= seuil) V2 = 0;
    if (V3 >= seuil) V3 = 0;

    // Logique de sélection originale
    if (V2 < 0.7 && V3 > 0.7) {

        return V3 / 33.242;
    }
    if (V1 < 0.7 && V2 > 0.7) {
        Serial.print("2");
        return V2 / 8.153;
    }
    if (V1 > 0.7) {
        Serial.println(V1);
        Serial.println(V2);
    }
}
```

```
        Serial.println(V3);  
        return v1 / 2.0;  
    }  
  
    return 0;  
}
```